

NGINX, Wireshark, & OpenSSL

Build I.T. Club
September 30, 2026

Web Servers, Packet Analyzers, & Encryption

Build I.T. Club
September 30, 2026

web servers: overview

- web server - computer that accepts HTTP requests from a client and returns a response with the requested content
- basic functionality - essentially all web servers are set up with the ability to do static content serving over HTTP and log traffic. modern web servers typically expand on this with significantly more advanced functionality

https://en.wikipedia.org/wiki/World_Wide_Web
https://en.wikipedia.org/wiki/Web_server



first web server, CERN, 1990. although file transfer protocols existed well before this, this was the first time we saw now-ubiquitous HTTP/HTML/URLs and networked hyperlinks



NGINX

web servers: NGINX

- NGINX is free and open-source web server software (NGINX can also be configured to do other things, but we're restricting our attention to the web server part)
- currently, a plurality of all websites are accessed through NGINX web servers.

<https://en.wikipedia.org/wiki/Nginx>

<https://suddenlysixam.club/guides/2025-04-07-nginx/>

<https://nginx.org/>

<https://webmasters.stackexchange.com/questions/139506/what-is-the-bare-minimum-to-host-a-website-without-pre-packaged-solutions>

packet analysis: Wireshark



- free and open-source packet analyzer software
- go beyond NGINX's built-in logging capabilities - NGINX is higher-level software than Wireshark, which looks at raw data packets

<https://en.wikipedia.org/wiki/Wireshark>

https://en.wikipedia.org/wiki/Packet_analyzer

navigate to localhost in your web browser

```
sudo apt install vim
```

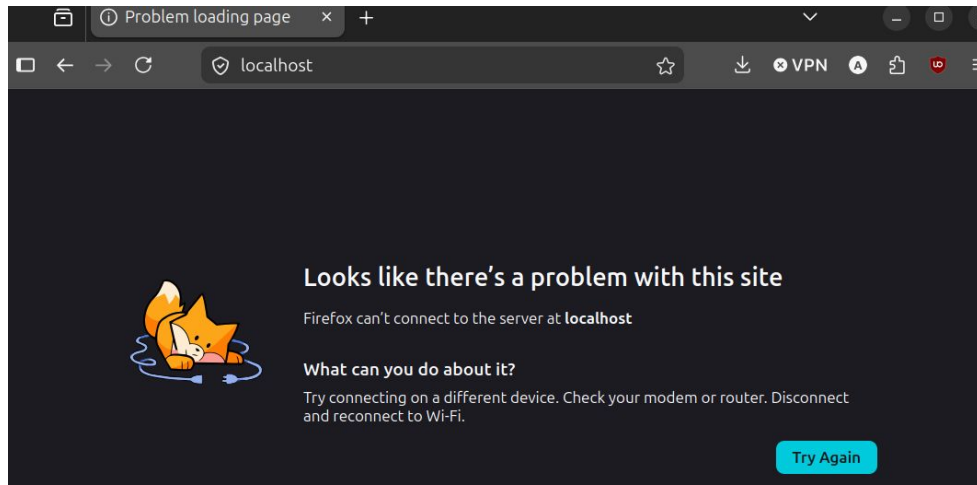
```
sudo apt install nginx
```

```
sudo apt install wireshark
```

 (Wireshark was added to default repositories in Debian 13) -> select "No"

```
sudo apt update && sudo apt upgrade
```

```
sudo wireshark
```



select 'any' to start capturing packets

navigate to localhost or localhost/?user=<user> in web browser - look for HTTP request "GET /?user=<user> HTTP/1.1"

try visiting a website or using a search engine to make a web query and watch the traffic in Wireshark!

The image shows a web browser window on the left and a Wireshark packet capture window on the right. The browser window displays the 'Welcome to nginx!' page, which includes the text: 'If you see this page, the nginx web server is successfully installed and working. Further configuration is required. For online documentation and support please refer to nginx.org. Commercial support is available at nginx.com. Thank you for using nginx.'

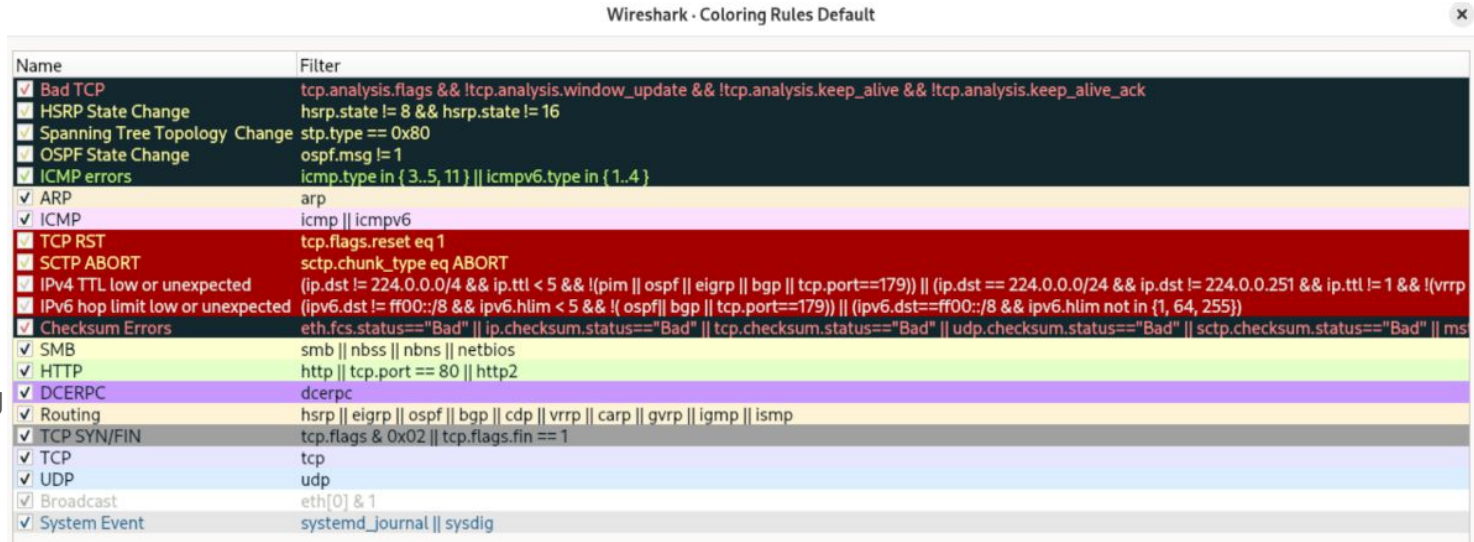
The Wireshark window is titled 'Capturing from any' and shows a list of captured packets. The first packet is selected, showing details of an HTTP GET request to localhost.

No.	Time	Source	Destination	Protocol	Length
1	0.000000000	127.0.0.1	127.0.0.1	TCP	76
2	0.000016115	127.0.0.1	127.0.0.1	TCP	76
3	0.000029967	127.0.0.1	127.0.0.1	TCP	68
4	0.000194872	127.0.0.1	127.0.0.1	HTTP	584
5	0.000203073	127.0.0.1	127.0.0.1	TCP	68
6	0.000305888	127.0.0.1	127.0.0.1	HTTP	241
7	0.000310146	127.0.0.1	127.0.0.1	TCP	68

select 'any' to start capturing packets

navigate to localhost or localhost/?user=<user> in web browser - look for HTTP request "GET /?user=<user> HTTP/1.1"

try visiting a website or using a search engine to make a web query and watch the traffic in Wireshark!



Wireshark coloring rules (View -> Coloring rules):

the HTML for the NGINX default website is
at /var/www/html/index.nginx-debian.html
- vandalize it a bit if you wish!

(back up beforehand with `sudo cp`
`index.nginx-debian.html`
`index.nginx-debian.html.bak`)

try visiting others' websites!

Welcome to nginx!

If you see this page, you should **LOVE**
YOURSELF!

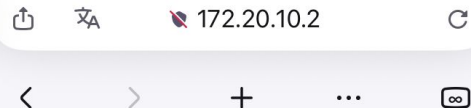
Despite everything, it's still you.

For online documentation and support please
refer to nginx.org.

Commercial support is available at nginx.com.

Thank you for using nginx.

Connected to modified
NGINX default website
from cell phone on the
same wireless network



HTTP/HTTPS

- try to visit `https://localhost` in your web browser
- if you were connected to a public wireless network, a malicious actor could use Wireshark (in monitor mode) to read everything you sent back and forth - clearly problematic!
- HTTPS is generally default in 2026
- “HTTP over TLS” - encryption, ID, data integrity
- port 80 vs. port 443

<https://en.wikipedia.org/wiki/HTTPS>

<https://www.geeksforgeeks.org/computer-networks/difference-between-http-and-https-2/>

OpenSSL



- TLS (Transport Layer Security, 1999) vs. SSL (Secure Socket Layer, 1994) - TLS builds on SSL and is more secure.
- SSL has been deprecated for a long time, but “SSL” is still in common usage to refer to TLS
- OpenSSL is open-source command line software

<https://www.geeksforgeeks.org/computer-networks/difference-between-secure-socket-layer-ssl-and-transport-layer-security-tls/>

https://en.wikipedia.org/wiki/Transport_Layer_Security

https://en.wikipedia.org/wiki/Message_authentication_code

`openssl genrsa -out localhost.key 2048` (private key - necessary for encryption! For production, encrypt out of plain text with `-aes256`)

`openssl req -key localhost.key -new -out localhost.csr`
(signing request - leave everything blank)

`openssl x509 -signkey localhost.key -in localhost.csr -req -days 365 -out localhost.crt` (self-signed certificate - max cert lifetime is currently 200 days)

so now we have a TLS certificate that's signed with a private key. But...

>>> who's signing the certificates? <<<

```
openssl req -x509 -sha256 -days 1000 -newkey rsa:2048  
-keyout rootCA.key -out rootCA.crt (self-signed root CA -  
random password; leave options blank)
```

```
openssl x509 -req -CA rootCA.crt -CAkey rootCA.key -in  
localhost.csr -out localhost.crt -days 365 -CAcreateserial  
(sign CSR with root CA certificate/key)
```

```
sudo cp localhost.crt /etc/ssl/certs/
```

```
sudo cp localhost.key /etc/ssl/private/
```

```
sudo vim /etc/nginx/sites-enabled/default (comment out  
listen 80 lines; uncomment listen 443 lines. then add under  
the listen statements:
```

```
ssl_certificate      /etc/ssl/certs/localhost.crt;
```

```
ssl_certificate_key  /etc/ssl/private/localhost.key;)
```

```
sudo nginx -t (Check that your NGINX config file is valid)
```

```
sudo systemctl restart nginx
```

>>> now you can connect to https://localhost!! <<<

hop back on Wireshark and navigate to https://localhost - you will see "TLSv1.3", etc., in the Protocol column - your traffic is encrypted!

real production environments - restrict to TLSv1.2/v1.3, more secure ciphers than default, dynamic content serving, virtual hosting, content cache, session tokens, authorization, etc.

<https://www.baeldung.com/ops/nginx-http-https-domains-redirection>

questions/comments/concerns